BB84

Rendszerarchitektúra, kvantumfizikai modell, felhasználói kézikönyv és tesztelési dokumentáció

KÉSZÍTETTE: Lipóczi Levente László (SUSXSW) • Fekete Botond (VZ6YKQ)

1. Bevezetés és a BB84 protokoll elméleti háttere

A Charles H. Bennett és Gilles Brassard által 1984-ben publikált **BB84 protokoll** az első és legelterjedtebb kvantumkulcsszétosztó (Quantum Key Distribution – QKD) eljárás. Célja, hogy két távoli fél – a kriptográfiai konvencióknak megfelelően **Alice** (küldő) és **Bob** (fogadó) – között egy nyilvános kvantumcsatorna és egy hitelesített klasszikus csatorna segítségével olyan közös, titkos bitsorozatot (kulcsot) hozzon létre, amelynek biztonságát nem matematikai problémák számítási bonyolultsága, hanem közvetlenül a kvantummechanika alaptörvényei garantálják.

A protokoll biztonsága két fundamentális kvantumfizikai elven nyugszik: a **klónozhatatlansági tételen** (No-Cloning Theorem), amely kimondja, hogy egy ismeretlen kvantumállapotról nem készíthető tökéletes másolat, valamint a **kvantummérés állapotromboló jellegén**, amely szerint egy szuperpozíciós állapot mérése a választott mérési bázis valamelyik sajátállapotába kényszeríti a rendszert. Ha egy harmadik fél, **Eve** (a lehallgató) megkísérli elfogni és megmérni a fotonokat (Intercept-Resend támadás), elkerülhetetlenül mérési hibákat visz be az átvitelbe, amelyet Alice és Bob a nyilvános ellenőrző mintavétel során statisztikailag kimutatnak.

1.1. Polarizációs bázisok és kvantumállapotok

A szimulátor az ideális, egyfotonos polarizációs BB84-modellt valósítja meg. Alice minden egyes elküldendő klasszikus bithez ($b \in \{0, 1\}$) választ egy előkészítési bázist a két egymással komplementer (nem ortogonális, hanem egymással 45°-os szöget bezáró) bázis közül:

- **Rektlineáris (Z) bázis:** A 0 bitet a vízszintes polarizációjú $|0\rangle$ (0°) állapotba, az 1 bitet a függőleges polarizációjú $|1\rangle$ (90°) állapotba kódolja.
- **Diagonális (X) bázis:** A 0 bitet a $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ (−45°) állapotba, az 1 bitet a $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$ (+45°) állapotba kódolja.

Bázis jele	Bázis megnevezése	Klasszikus bit = 0	Klasszikus bit = 1	UI megjelenítés (polarizációs szög)
Z	Rektlineáris (Horizontális / Vertikális)	$\|0\rangle$ (Vízszintes)	$\|1\rangle$ (Függőleges)	0° (vízszintes nyíl) / 90° (függőleges nyíl)
X	Diagonális (Átlós, Hadamard-transzformált)	$\|+\rangle$ (−45° átlós)	$\|-\rangle$ (+45° átlós)	−45° (átlós nyíl) / +45° (átlós nyíl)

1.2. A kvantummérés matematikája és az Intercept-Resend támadás (QBER)

Amikor egy $B_k \in \{Z, X\}$ bázisban előkészített kvantumbitet egy $B_m \in \{Z, X\}$ bázisban mérünk meg, két eset lehetséges:

1. **Egyező bázis ($B_k = B_m$):** A mérés determinisztikus, $P = 1$ valószínűséggel az eredetileg kódolt bitet adja vissza, és az állapot nem változik.

2. **Eltérő bázis ($B_k \neq B_m$):** A kvantumbit a mérési bázis két sajátállapotának egyenlő súlyú szuperpozíciójában van, így a mérés kimenetele teljesen véletlenszerű: $P(0) = 0{,}5$ és $P(1) = 0{,}5$.

Ha **Eve aktív**, és minden átküldött fotont megmér a saját B_E bázisában, majd a mért bitnek megfelelő állapotot újraküldi Bobnak, akkor azokban a pozíciókban, ahol Alice és Bob azonos bázist választott ($B_A = B_B$, azaz a szűrés során megtartott pozíciókban):

- 50% eséllyel Eve is ugyanazt a bázist választja ($B_E = B_A$), így nem okoz hibát ($P_{hiba} = 0$).
- 50% eséllyel Eve téves bázist választ ($B_E \neq B_A$), amivel lerombolja az eredeti állapotot. Ekkor Bob 50% eséllyel a helyes, 50% eséllyel a hibás bitet méri.

$$QBER_{elméleti\ (aktív\ Eve)} = P(B_E \neq B_A) \times P(\text{Bob hibás bitet mér} \mid B_E \neq B_A) = 0{,}5 \times 0{,}5 = 0{,}25\ (25\%)$$

A **QBER (Quantum Bit Error Rate – kvantum bithibaarány)** az ellenőrző mintában felfedett bitek közötti eltérések aránya. Ideális, zajmentes csatornán lehallgatás nélkül $QBER = 0\%$, míg teljes Intercept-Resend támadás mellett a QBER várható értéke **25%** körül mozog.

2. Rendszerarchitektúra és felhasznált könyvtárak

A szoftver három, egymástól tisztán elkülönített rétegre bomlik (domain modell, állapotgép/vezérlő, valamint webes API és prezentációs réteg). Ez a felépítés biztosítja, hogy a kvantumprotokoll számításai mellékhatásoktól mentesen, önállóan is tesztelhetők legyenek, miközben a felhasználói felület mind helyi Python-szerverről, mind szerver nélküli (standalone) módban futtatható.

Fájl / Modul	Réteg	Elsődleges felelősségi kör
`bb84.py`	Mag (Domain modell)	Elemi kvantummérés (`meres`), pozíciók előkészítése (`elokeszites`), Eve és Bob mérései (`eve_merese` , `bob_merese`), bázisegyeztetés (`szures`), ellenőrző mintavétel (`mintavetel`), `qber` és `kulcsjeloltek` számítása.
`controller.py`	Futásvezérlő (Állapotgép)	A `FutasVezerlo` osztály: kezeli a protokoll 6 fázisát, az egyenkénti (`kovetkezo`) és fázisonkénti (`osszes_merese`) léptetést, valamint a következő mérendő qubit indexének meghatározását.
`app.py`	HTTP / JSON API	Flask webalkalmazás, szálbiztos (`RLock`) munkamenet-kezelés, fázisonkénti mélymásolat (`deepcopy`) alapú előzménynaplózás, bemenetvalidáció és hibakezelés.
`templates/index.html` `static/app.js` , `style.css`	Felhasználói felület (UI/UX)	Reszponzív laborfelület, interaktív SVG polarizációs szimbólumok, animált kvantumcsatorna, fázisnapló-időgép, pozíciótáblázat és kísérletkonfigurációs dialógusablak.
`BB84_Standalone_Szimulator.html` `inditas_windows.bat`	Standalone futtatható csomag	Böngészővel rendelkező Windows-os gépen telepítés nélkül, dupla kattintással indítható önálló alkalmazás (beágyazott betűtípusokkal, stíluslapokkal és lokális BB84 motorral).
`tests/test_*.py`	Automatizált tesztkészlet	

Fájl / Modul	Réteg	Elsődleges felelősségi kör
		60 darab egység- és integrációs teszt (`test_bb84.py` , `test_controller.py` , `test_app.py`), determinisztikus véletlenszám-mockolással.

2.1. Felhasznált könyvtárak és technológiák

- **Python 3.10+ szabványos könyvtárak:**
  - `random`: A kvantummérés eltérő bázisú bitválasztásának (`random.choice((0, 1))`) és az ellenőrző minta ismétlés nélküli kiválasztásának (`random.sample`) szimulációja.
  - `math`: Az ellenőrző mintaméret felfelé kerekítése (`math.ceil`), amely biztosítja, hogy pozitív százalék és nem üres szűrt sorozat esetén legalább 1 bit ellenőrzésre kerüljön.
  - `copy.deepcopy`: A fázisok pillanatképeinek elmentése a memóriában, hogy a korábbi fázisok visszanézésekor a későbbi mérések ne írhassák felül a múltbeli állapotokat.
  - `threading.RLock` , `secrets` , `uuid.uuid4`: Szálbiztos hozzáférés és kriptográfiailag biztonságos munkamenet-azonosítás.
  - `unittest` és `unittest.mock.patch`: Teljes körű automatizált tesztelés.
- **Külső Python könyvtárak (`requirements.txt`):**
  - `Flask==3.1.3` (és a vele érkező `Werkzeug`): Könnyűsúlyú WSGI webkeretrendszer a REST-jellegű JSON végpontok és a sablonkiszolgálás megvalósításához.
  - `waitress==3.0.2`: Produkciós szintű, tiszta Python WSGI HTTP-szerver, amely Windows és Linux környezetben egyaránt stabilan futtatható a helyi `127.0.0.1:8765` címen.
- **Frontend technológiák és tipográfia:**
  - Szabványos **HTML5** (natív `<dialog>` modális ablak, ARIA akadálymentesítési attribútumok), **CSS3** (CSS változók, Grid/Flexbox elrendezés, kulcskockás fotonanimációk) és keretrendszermentes **ES6+ JavaScript** (Web Crypto API `crypto.getRandomValues` a kriptográfiai minőségű véletlen bemenetgeneráláshoz).
  - Helyileg tárolt, nyílt forráskódú (SIL Open Font License) változó betűtípusok: **Anybody.ttf** (karakteres, technikai kijelzőbetűtípus a fejlécekhez és fázisszámokhoz) és **Commissioner.ttf** (kiváló olvashatóságú szövegbetűtípus).

3. A program működésének részletes bemutatása

3.1. Adatszerkezet és elemi műveletek (`bb84.py`)

A szimuláció alapegysége a kvantumbit-pozíció, amelyet a program egy egységes szerkezetű Python-szótárként (dictionary) tárol. Egy N qubitből álló kísérlet egy N elemű listát (`poziciok`) alkot:

```
{
    "alice": ("Z", 0),          # (Előkészítési bázis, Eredeti bit)
    "eve": ("X", None),         # None ha Eve inaktív; (Eve bázisa, None) mérés előtt; (Eve
bázisa, mért bit) mérés után
    "bob": ("Z", None),         # (Bob bázisa, None) mérés előtt; (Bob bázisa, mért bit) mérés
után
    "megtartva": None,          # None szűrés előtt; True ha Alice és Bob bázisa azonos,
különben False
    "ellenorzo_minta": False    # True, ha a pozíciót nyilvánosan felfedték a QBER
ellenőrzéséhez
}
```

A modul függvényei szigorú előfeltétel-ellenőrzést (defenzív programozást) alkalmaznak:

- Az `elokeszites()` ellenőrzi, hogy a listák nem üresek-e, azonos hosszúságúak-e, a bitek kizárólag `0` vagy `1` egész számok, a bázisok pedig kizárólag `"Z"` vagy `"X"` karakterláncok-e. Bármilyen eltérés esetén beszédes `ValueError` kivételt dob.
- Az `eve_merese()` és `bob_merese()` idempotens módon működik: a már megmért (`bit is not None`) pozíciókat ismételt híváskor sem sorsolja újra, így egy véletlen `0`-s mérési eredmény is biztonságosan megőrződik. Emellett megakadályozzák a sorrendiség megsértését (aktív Eve esetén Bob nem mérhet Eve előtt, és Eve nem írhatja felül Bob már elvégzett mérését).
- A `mintavetel()` a megtartott pozíciók `math.ceil(len(megtartott) * szazalek / 100)` hányadát jelöli meg ellenőrző mintaként. Ha már történt mintavétel, újabb híváskor nem sorsol új mintát, megakadályozva a nyilvánosságra hozott bitek manipulálását.
- A `kulcsjeloltek()` kizárólag a `megtartva == True` és `ellenorzo_minta == False` pozíciókból gyűjti össze Alice és Bob végső bitjeit, hiszen a nyilvánosan egyeztetett mintabitek már nem használhatók titkos kulcsként.

3.2. Fázisvezérlés és léptethetőség (`controller.py` és `app.py`)

A `FutasVezerlo` osztály a protokoll menetét 6 jól elkülönülő fázisra bontja:

1. **`elokeszites` (01 – Előkészítés):** Alice állapotai és a résztvevők bázisválasztásai bekerülnek a rendszerbe, de mérés még nem történt.
2. **`eve_merese` (02 – Eve mérése):** Csak aktív támadó esetén érhető el (kikapcsolt Eve mellett a vezérlő automatikusan átugorja). A *Következő* gomb hatására Eve egyszerre pontosan egyetlen qubitet mér meg, míg az *Összes kvantumbit mérése* gomb a fázis összes hátralévő qubitjét megméri.
3. **`bob_merese` (03 – Bob mérése):** Bob qubitenként (vagy egy lépésben az összeset mérve) elvégzi a méréseit. Az utolsó qubit megmérése után a fázis még látható marad a képernyőn, és csak a következő kattintás lépteti tovább a protokollt.
4. **`szures` (04 – Bázisegyeztetés):** Alice és Bob nyilvánosan összehasonlítják bázisaikat; a táblázatban megjelölésre kerülnek a megtartott (`✓ Megtartva`) és elvetett pozíciók.
5. **`mintavetel` (05 – Ellenőrző minta):** A megtartott bitek megadott százaléka nyilvánosan felfedésre kerül (`◇ Felfedve`).
6. **`eredmenyek` (06 – Eredmények):** Kiszámításra kerül a mintabeli QBER érték (százalékos formában), valamint megjelenik Alice és Bob szűrt, maradék kulcsjelöltje.

> **FÁZISNAPLÓ („IDŐGÉP” FUNKCIÓ)**
> Az `app.py` minden művelet után `deepcopy` segítségével elmenti az adott fázis pillanatképét az `elozmenyek` szótárba. Ennek köszönhetően a felhasználó a bal oldali Fázisok sávban bármikor visszakattinthat egy korábbi fázisra (pl. megnézheti, milyen állapotban volt a rendszer a szűrés előtt), majd az *Élő futás* gombbal visszatérhet az aktuális lépéshez.

4. Telepítési és felhasználói útmutató (Windows / Standalone)

4.1. Indítási módok böngészővel rendelkező Windows-os gépen

A beadandó csomag két teljes értékű, egymással 100%-ban azonos felületű indítási módot biztosít Windows operációs rendszeren:

1. **Teljesen önálló, szerver nélküli (Standalone HTML) indítás – Nincs szükség Pythonra:** Nyissa meg közvetlenül bármelyik modern böngészőben (Edge, Chrome, Firefox) a `BB84_Standalone_Szimulator.html` fájlt (dupla kattintással). Ez az egyetlen fájl tartalmazza a beágyazott betűtípusokat, a teljes stíluslapot és a szimulátor helyi motorját, így bármilyen előzetes telepítés vagy internetkapcsolat nélkül azonnal működik.
2. **Automatikus indítóparancsfájl (`inditas_windows.bat`) vagy Python szerver:** Dupla kattintással elindítható az `inditas_windows.bat` fájl is: ha a gépen van telepítve Python, automatikusan létrehozza a virtuális környezetet, telepíti a `Flask` és `waitress` csomagokat, elindítja a helyi szervert a `http://127.0.0.1:8765` címen, és megnyitja a böngészőt. Ha a gépen nincs Python, a parancsfájl automatikusan megnyitja a `BB84_Standalone_Szimulator.html` fájlt. Parancssorból történő manuális indításhoz:

```
python -m pip install -r requirements.txt
python app.py
# Majd nyissa meg a böngészőben: http://127.0.0.1:8765
```

4.2. A felhasználói felület (UI) kezelése lépésről lépésre

1. **Kísérlet beállítása:** A bal felső sarokban található *Kísérlet beállítása* gombra kattintva nyílik meg a konfigurációs ablak. Itt megadható:
   - **Qubitek száma (1–256):** A csatornán átküldendő fotonok darabszáma.
   - **Ellenőrző minta (0–100%):** A szűrt bitek hány százalékát áldozzák fel a lehallgatás ellenőrzésére.
   - **Véletlen kitöltés gomb:** A megadott qubitszámnak megfelelően kriptográfiai véletlenszám-generátorral tölti ki Alice bitjeit (`0/1`), valamint Alice, Bob és Eve bázisait (`Z/X`).
   - **Manuális bit- és bázismegadás:** A szövegmezőkben a felhasználó szabadon átírhatja vagy begépelheti a kívánt `0/1` és `Z/X` sorozatokat (az olvashatóság kedvéért szóközök és kisbetűk is használhatók).
   - **Eve aktív jelölőnégyzet:** Egyetlen kattintással be- vagy kikapcsolható a lehallgató a csatornán.
2. **Kvantumcsatorna és polarizációs vizsgálat:** A középső színpadon (Stage) három kártya reprezentálja **Alice**, **Eve** és **Bob** állomását. Az egyes kártyákon dinamikus SVG-grafikon mutatja az aktuálisan kiválasztott qubit polarizációs szögét (0°, 90°, −45°, +45°). Amikor Eve vagy Bob mérést végez, a kártyák közötti csatornán animált fotonimpulzus halad végig.

3. **Léptetés:** A *Következő* gombbal a szimuláció qubitről qubitre, majd fázisról fázisra léptethető. A mérési fázisokban az *Összes kvantumbit mérése* gombbal az adott résztvevő összes hátralévő mérése egyetlen kattintással elvégezhető.
4. **Interaktív Pozíciók táblázat:** Az alsó táblázat valós időben mutatja minden egyes qubit állapotát. Bármelyik sorra (vagy a pozíció sorszámára, pl. `01`, `02`) kattintva a felső kvantumcsatorna-ábra azonnal az adott qubit polarizációs állapotait jeleníti meg. A táblázat külön oszlopban, piros `≠ Eltérés` jelzéssel emeli ki azokat az egyező Alice–Bob bázisú pozíciókat, ahol Eve beavatkozása miatt bithiba keletkezett.

5. Automatizált egység- és integrációs tesztek

A projekt a `tests/` könyvtárban három tesztmodult tartalmaz (`test_bb84.py` , `test_controller.py` , `test_app.py`), amelyek a `py -m unittest discover -s tests -v` paranccsal futtathatók. A tesztek az `unittest.mock.patch` segítségével vezérlik a véletlenszám-generálást, így determinisztikusan ellenőrzik az összes határesetet: az azonos és eltérő bázisú méréseket, az ismételt hívások idempotenciáját, a hibás bemenetek visszautasítását, a fázisváltások szigorú sorrendjét és a különálló böngészőmunkamenetek izolációját.